

claude-windows / 02b9e9c9-3806-45ca-9b24-2ecc35a81efd

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-muneem\02b9e9c9-3806-45ca-9b24-2ecc35a81efd\subagents\workflows\wf_27df53c8-8e0\agent-a5785411d62c14b6c.jsonl`
- SHA-256: `5ee25daccfeb946611d0ba3a7985ddb0be021869ceb610315ae344ae37c09b70`
- Source modified: `2026-06-10T16:18:33+00:00`
- Imported at: `2026-07-05T16:48:21+00:00`
- project: `wf_27df53c8-8e0`
- session_id: `02b9e9c9-3806-45ca-9b24-2ecc35a81efd`
```

Transcript

1. user (2026-06-10T16:13:27.798Z)

You are auditing the ARCHITECTURE, VISION, ROADMAP, and product posture of gmail-attachments-to-gdrive (C:/Users/avidu/Projects/gmail-attachments-to-gdrive) ? an Apps Script project whose stated north star is 'turn a person's email into a high-fidelity, structured, queryable record of their life', positioned as the extraction/ingest front door that hands minimized signal to downstream processors (muneem for finance; future siblings for grocery/travel/coupons).

Read COMPLETELY: docs/VISION.md, docs/ARCHITECTURE.md, ROADMAP.md, README.md, PRIVACY.md, LICENSE, appsscript.json, package.json. Skim src/ to verify doc claims (e.g. the egress-guard test tests/auditability.test.ts, executeAs in appsscript.json, scopes requested).

Assess:

1. The phased plan (0 ingest model ? 1 GAS-only classify/extract/index ? 2 coverage/re-extraction ? 3 handoff to hosted services ? 4 surfaces/ops): is each phase realistic ON APPS SCRIPT? Specifically: classification + extraction + OCR of PDF attachments inside GAS quotas (6 min/run, 90 min/day triggers, ~20k Gmail reads/day, no native libs ? how would PDF OCR even run?); where does the index live (Sheets? size limits?); what does 'versioned handoff contract' mean concretely and does ANY artifact define it yet?
2. The auditability invariant (this repo is the only Gmail-touching code, self-deployable, no egress): is it enforceable as stated once Phase 3 (egress to hosted service) lands? Is 'logs to Cloud Logging' already egress? Does the multi-user web app weaken it?
3. Privacy/PII reasoning: bank statements ARE PII ? can the 'only non-PII signal leaves' rule and the muneem handoff coexist? Where would redaction of a bank statement PDF happen and is that feasible in GAS? Is PRIVACY.md consistent with VISION.md and with what the code does today?
4. Product posture: 'build for public, stay under 100 users (OAuth Testing mode)' ? check README's multi-user/publishing section; what's the real cost/process wall (gmail.readonly is a RESTRICTED scope ? annual CASA security assessment) and is the plan's 'decide later' stance coherent? What breaks at user 101?
5. Roadmap state: ROADMAP.md is mostly checked-done items plus a thin tail ? what SHOULD the next roadmap items be to serve the vision (the Phase-1 extractor framework? the handoff contract? SECURITY.md?). Is there any item tracking the grocery-receipts vertical the VISION names as first?
6. Architecture doc quality: decision log hygiene, anything contradicting muneem's docs (muneem has its own Gmail OAuth ingestion and a planned 'muneem fetch' ? VISION.md here says downstream never reads Gmail).

Return the structured report. The 'map' field must compactly state the vision, the phase plan, the auditability invariant, current implemented surface, and the publishing constraints as the docs state them. Structured output only.

2. user (2026-06-10T16:13:33.516Z)

```
1      # Vision ? Email as a smart way to organize your life
2
3      > Status: living document. The overall product direction. Architectural detail
4      > and the decision log live in [ `ARCHITECTURE.md` ] (./ARCHITECTURE.md).
```

```

5
6    ## North star
7
8    Turn a person's email into a high-fidelity, structured, queryable record of
9    their life ? receipts, bank statements, flights & travel, coupons, shopping ?
10   built automatically, privately, and reliably. Email becomes the source of truth
11   that organizes the rest.
12
13   ## What this repo is ? and is not
14
15   This repository (`gmail-attachments-to-gdrive`) is the extraction / ingest
16   layer. It is the reliable, quota-aware "front door" that:
17
18   - watches a user's Gmail in their own account,
19   - archives raw artifacts (attachments) ? still valuable on its own,
20   - classifies messages and extracts a clean, normalized signal,
21   - redacts/minimizes that signal, and hands it off to downstream
22     processors.
23
24   It is not the domain parser, the datastore, or the analytics engine. That
25   logic is deliberately isolated in separate downstream services:
26
27   | Concern | Owner |
28   | --- | --- |
29   | Gmail watching, archiving, classification, extraction, redaction, handoff | this
30   repo |
31   | Bank-statement & financial parsing, indexing, encrypted storage, analytics |
32   Muneem (`avidullu/muneem`) |
33   | Grocery / receipts, coupons, travel processing | future siblings of Muneem |
34
35   The boundary is intentional: extraction (this repo) is decoupled from
36   processing (downstream). This repo never imports domain-parsing logic;
37   downstream services never read Gmail. They meet only at a versioned handoff
38   contract.
39
40   ## Foundation first
41
42   Attachment archiving is the hardened core. Everything else is built on its
43   proven strengths, which we preserve and extend, never bypass:
44
45   - per-user isolation (`executeAs: USER_ACCESSING`),
46   - a per-user lock and self-healing trigger fan-out,
47   - the hybrid `history.list` + date-window sync,
48   - "mark a message done only on clean completion,"
49   - pure-logic / GAS-shell separation with heavy unit testing,
50   - ruthless respect for Apps Script quotas and the 6-minute boundary.
51
52   ## Audience & rollout
53
54   - We build for public ? the long-term goal is a wide launch.
55   - For now, stay under 100 users (OAuth "Testing" mode) to harden the
56     system, learn the value proposition, and make an educated call on whether to
57     financially invest (the annual CASA security assessment, hosted infra) once
58     there is a substantial user base.
59   - Until that call, we stay in Google's no-verification zone (see
60     [`../README.md`](`../README.md`) ? Multi-user deployment / publishing).
61
62   ## Privacy north star
63
64   - Minimize egress. Only non-PII signal ever leaves the user's Google
65     account, and only to our own isolated, hosted service ? never third
66     parties.
67   - Scrub + encrypt as required, applied before anything leaves.
68   - Continuously reduce what leaves while maintaining product health.
69   - The in-account baseline needs zero external dependency; external

```

```

68     processing is a deliberate, **separately-consented** upgrade with a privacy
69     policy that names exactly what is sent, where, and why.
70
71     ### The auditability invariant
72
73     **This repo is the only code that ever touches a user's Gmail, and it stays
74     auditable and self-deployable by a power user at any time.** It is intended to be
75     open-sourced precisely so that a privacy-conscious user can read ? and run ? the
76     exact code that reads their mail. Muneem and every downstream service operate
77     only on minimized, non-PII signal and never hold the Gmail grant. This is a
78     binding constraint on all future work; the enforceable form (and a per-change
79     checklist) lives in [`ARCHITECTURE.md`](./ARCHITECTURE.md#auditability-invariant-binding).
80
81     ## First verticals
82
83     1. **Grocery receipts** ? high volume, forgiving fidelity bar; proves the whole
84       ingest ? classify ? extract ? index pipeline cheaply.
85     2. **Bank statements** ? high-fidelity path; output feeds **Muneem**, proving the
86       redact ? handoff ? downstream-processing seam.
87
88     ## Success criteria (proposed)
89
90     - The user's index auto-populates with correct entities at **high precision**;
91       low-confidence items are **surfaced for review**, never silently dropped.
92     - Re-running an improved extractor over history **corrects past entities without
93       duplicates** and without re-copying blobs.
94     - Egress stays **minimal and non-PII**; the in-account baseline works with no
95       external service available.
96
97     ## Phased plan (high level)
98
99     Detailed status is tracked in [`../ROADMAP.md`](../ROADMAP.md); the technical
100    shape of each phase is in [`ARCHITECTURE.md`](./ARCHITECTURE.md).
101
102    0. **Ingest model** ? a rich, stable `EmailMessage`; the copier consumes it
103      (no behavior change).
104    1. **Classify + extract + index (GAS-only, in-account)** ? classifier, pluggable
105      extractors, normalized entities, a user-owned index. No egress.
106    2. **Coverage + re-extraction** ? more extractors, body/OCR handling, extraction
107      cursors & status, "re-extract for new rules," review queues.
108    3. **Handoff to hosted services** ? redact/minimize, then feed Muneem & siblings;
109      separate consent + privacy update; extraction/egress budgets.
110    4. **Surfaces & ops** ? dashboards, Calendar/Tasks automation, a richer review &
111      configuration UI, Shared Drives, schema migration.
112

```

3. user (2026-06-10T16:13:34.483Z)

```

1     # Architecture
2
3     > Status: living document. Captures the **target architecture** for the
4     > extraction pivot and the **decisions** behind it. Product framing is in
5     > [`VISION.md`](./VISION.md). This supersedes the original "just copy
6     > attachments" mental model; the copier survives as one extractor at the leaf.
7
8     ## Auditability invariant (binding)
9
10    **This repository is the _only_ component that reads or processes a user's
11    Gmail, and it must remain auditable and self-deployable by a power user at any
12    time.** This is the product's core privacy guarantee and a deliberate constraint
13    on every future decision (see ADR-008). It is why the project is built to be
14    open-sourced: *don't trust us ? read, and run, the code.*
15
16    Every change MUST preserve all of the following. A change that cannot is, by

```

```

17 definition, a decision to break the invariant and needs explicit owner sign-off.
18
19 1. **Sole Gmail surface.** Gmail is read/processed only here, in the user's own
20 account (`executeAs: USER_ACCESSING`). No other component ? Muneem included ?
21 ever holds the Gmail OAuth grant or sees raw mail. Muneem may orchestrate
22 this app; it is never itself the Gmail reader.
23 2. **Egress == the audited boundary.** Nothing derived from a user's mail leaves
24 their Google account except through the in-repo **REDACT/MINIMIZE** stage,
25 carrying only minimized, non-PII signal, only to the user's own hosted
26 service, never to third parties. The set of egress destinations is explicit
27 and reviewable in source.
28 3. **Run-your-own-from-source.** A power user can build and deploy their own
29 instance from the published source and obtain an equivalent app; the build is
30 reproducible and the verification steps are documented ? "trust by running the
31 code you read," not merely "read the code."
32 4. **No secrets in source.** Credentials exist only as deploy-time secrets; the
33 repository contains nothing sensitive.
34 5. **Transparent over clever** at the read and egress/redaction boundaries ?
35 prefer obviously-correct, reviewable code even at some cost to brevity.
36
37 **PR review checklist (apply to every change):**
38
39 - [ ] Gmail access stays entirely within this repo, in-account?
40 - [ ] If any data leaves the account, does it pass through REDACT/MINIMIZE and go
41 only to the user's own service (non-PII)?
42 - [ ] Build still reproducible and self-deployable from source?
43 - [ ] No new secrets, third-party endpoints, or network calls? (Default: none ?
44 enforced by `tests/auditability.test.ts`.)
45
46 ## Principles (preserved from today's codebase)
47
48 1. **Shells around pure functions.** GAS-facing code is a thin shell; all logic
49 is pure and unit-tested.
50 2. **Quota discipline.** Respect the 6-minute execution limit, the 90-min/day
51 trigger budget, and per-user caps. Self-healing trigger fan-out; per-user
52 lock; "mark done only on clean completion."
53 3. **Per-user isolation.** Runs as the accessing user; state is per-user.
54 4. **Versioning & provenance everywhere.** Every persisted shape carries a
55 `schemaVersion`; every extracted entity carries where it came from and which
56 extractor (and version) produced it.
57
58 ## The pipeline
59
60 This repo owns the left half; downstream services own the right half. They meet
61 at the **handoff contract** only.
62
63 ```
64
65 Gmail ??? INGEST ??? CLASSIFY ??? EXTRACT ??? REDACT/MINIMIZE ??? HANDOFF ??? Muneem
66 /
67 (rich (type + (normalize (strip PII, (sink) grocery
68 /
69 EmailMsg) confidence) light) encrypt) coupon
70 svc
71 ? ??
72 parse,
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942

```

```

74 - **INGEST** builds a durable `EmailMessage` **once** per message (body, subject,
75 labels, attachment metadata ? via the Gmail advanced service). Extraction
76 re-runs over this record without re-touching Gmail or re-copying blobs.
77 - **CLASSIFY** assigns a type + confidence (sender/subject/body rules now; richer
78 models downstream later).
79 - **EXTRACT** is a registry of pluggable, versioned extractors. The current
80 attachment copier becomes the `RawArchive` extractor.
81 - **REDACT/MINIMIZE** enforces the privacy north star before any egress.
82 - **HANDOFF** routes a normalized payload to the right downstream **sink**.
83
84 ## Data model
85
86 All pure shapes, versioned from day one.
87
88 ...
89 EmailMessage {                                // INGEST output ? the stable substrate
90     schemaVersion, id, threadId, historyId?,
91     from, to[], cc[], subject, date, labels[],
92     snippet, plainBody?, htmlBody?,
93     attachments: [{ index, name, contentType, sizeBytes,
94                     attachmentId/partId, sha256?, driveFileId? }],
95 }
96
97 ExtractedEntity {                                // EXTRACT output ? the queryable payload
98     schemaVersion, entityKey,                    // stable dedup key, e.g.
99                                                    // receipt:{merchant}:{total}:{date}
100     type, subtype, confidence,
101     fields,                                       // typed per kind (receipt, statement, ?)
102     provenance: { sourceMsgIds[], attachmentIndex?,
103                   extractor, extractorVersion, extractedAt, model? },
104     status: pending|extracted|low_confidence|needs_review|corrected,
105     links: { driveFileId?, ? },
106 }
107
108 HandoffPayload {                                // REDACT/MINIMIZE output ? what leaves
109     schemaVersion, entityKey, type, extractorVersion,
110     nonPiiFields,                                // minimized, scrubbed signal only
111     // encrypted in transit; PII stripped or tokenized
112 }
113 ...
114
115 - **Entity keys** give content-stable dedup (re-mailed receipts; cross-source
116 fusion of an email confirmation + a PDF ticket ? one entity).
117 - **Attachment `sha256`** enables content-addressable dedup of re-sent files.
118 - **Provenance + `extractorVersion`** make "this extraction is stale ? re-run"
119 possible and auditable.
120
121 ## The extraction ? processing boundary
122
123 - This repo produces a normalized, classified, **redacted** `HandoffPayload` per
124 detected document/entity and routes it to a downstream **sink**.
125 - Downstream services (Muneem for finance; siblings for groceries/coupons) own
126 domain parsing, indexing, **encrypted storage**, and analytics.
127 - **Contract is owned jointly, defined from our side first.** The exact
128 Muneem-facing schema and transport are **TBD and aligned with that repo**
129 (this repo's GitHub tooling is scoped to itself, so the contract is specified
130 here and reconciled with Muneem separately).
131 - Hard rule: this repo imports **no** domain-parsing logic; downstream reads
132 **no** Gmail.
133
134 ## Storage abstraction (decision: extensible, not baked-in)
135
136 A single `Store` / `Sink` interface; code depends only on the interface.
137
138 | Impl | When | Properties |

```

```

139 | --- | --- | --- |
140 | `GoogleSheetsStore` (user-owned "LifeIndex" sheet) | v1 | in-account, queryable, zero
infra, doubles as a first dashboard |
141 | `RemoteStore` (our hosted DB) / Muneem-API sink | later | the durable system of record
|
142
143 `PropertiesService` stays only for small per-user **control state** (cursors,
144 flags, counters) ? never the entity index.
145
146 ## Privacy & egress controls
147
148 - **Egress only to our own isolated, hosted service. No third parties.**
149 - A **Redactor/Minimizer** stage runs before any egress: strip/tokenize PII,
150 keep only non-PII signal, encrypt in transit (encrypted at rest downstream).
151 - North star: continuously **shrink** what leaves while keeping the product
152 healthy.
153 - The in-account tier (Phases 0?2) has **no egress** and needs no privacy
154 change. The external tier (Phase 3) is **flag-gated + separately consented**,
155 and updates the privacy policy to name exactly what is sent and where.
156 - **Cascade to be aware of:** sending restricted-scope Gmail data outside Google
157 is what makes Google's **CASA security assessment mandatory for a public
158 launch**. Deferred ? consistent with "stay <100 until the user base justifies
159 the investment."
160 - **Open tension to resolve (Phase 3 design):** bank-statement and receipt
161 signal (merchant, amounts, account identifiers) is often itself sensitive.
162 "Non-PII only" needs a concrete per-vertical definition of what is sent vs.
163 tokenized vs. kept fully in-account.
164
165 ## State & cursors
166
167 - Keep the hybrid `history.list` + date-window skeleton; **extend, don't
168 duplicate.**
169 - Add an **extraction-aware cursor** ("fully extracted up to X with
170 extractor-set V") distinct from the copy cursor, plus per-message extraction
171 status ? so a "re-extract window for new rules" reprocesses without
172 re-copying (content hash) or duplicating entities (entity keys).
173
174 ## Quota, cost & pacing
175
176 - **Separate budgets:** raw-copy cap (existing) vs. extraction cap vs.
177 egress/handoff cap.
178 - Per-item time budgets, **prioritization** (recent + high-confidence/trusted
179 senders first), and **deferral** of heavy items.
180 - Heavy ML/PDF work happens **downstream**, not in Apps Script ? GAS stays the
181 lightweight, reliable watcher/archiver/handoff.
182
183 ## Observability
184
185 - Per-**classifier** and per-**extractor** success / failure / ambiguity rates ?
186 not just "files added."
187 - Surfaced **review queues:** "matched no extractor" and "low confidence."
188 - Structured logs carrying `messageId`; extraction & handoff metrics.
189
190 ## Platform spectrum
191
192 1. **Pure-GAS conservative extractors** (regex, known formats `.ics`/`.csv`,
193 native Drive OCR via Doc conversion) for the easy ~70%, **in-account**.
194 2. **Hosted, isolated service** (Muneem pattern) for heavy/ML extraction,
195 storage, and analytics ? fed only the minimized, non-PII handoff.
196
197 ## Decision log
198
199 | # | Decision | Rationale |
200 | --- | --- | --- |
201 | ADR-001 | Pipeline (ingest ? classify ? extract ? redact ? handoff), not a copy loop |

```

Treat mail as structured data; make extraction re-runnable over a durable record |

202 | ADR-002 | Extraction (this repo) is decoupled from processing (Muneem & siblings) | Isolate domain logic + encrypted storage + analytics downstream; clean contract seam |

203 | ADR-003 | Storage via a `Store`/`Sink` interface ? Sheets v1, remote/Muneem later | Extensible by design; never baked into the code |

204 | ADR-004 | Egress: non-PII only, to our own isolated hosted service, no third parties; scrub + encrypt; minimize | Privacy north star; preserves trust; in-account baseline needs no egress |

205 | ADR-005 | Audience: <100 (Testing) now, public later; CASA deferred | Harden + learn value before investing in assessment/infra |

206 | ADR-006 | First verticals: grocery receipts + bank statements (? Muneem) | Volume + the high-fidelity handoff path |

207 | ADR-007 | Preserve current strengths (pure/shell, quotas, triggers, per-user, versioning/provenance) | The recent rewrite is exactly why this pivot is feasible |

208 | ADR-008 | ****Option A:**** this repo is the sole Gmail-touching component ? in-account, auditable, self-deployable; Muneem orchestrates but never holds the Gmail grant. Codified as the binding [auditability invariant](#auditability-invariant-binding). | Verifiable privacy is the product's moat and shrinks future verification scope; it concentrates the trust-critical surface in one open, in-account place. |

209

210 ## Open items for the next iteration

211

212 - ****Muneem handoff contract:**** schema + transport (e.g. `UrlFetch` to a signed endpoint vs. write-to-a-shared-store) ? define jointly with `avidullu/muneem`.

213

214 - ****Per-vertical "non-PII" definition**** for receipts and bank statements (what is sent vs. tokenized vs. kept in-account).

215

216 - ****First extractor slice**** to build in Phase 1 (recommend a narrow

217 "receipt-smelling" detector) and its golden-email test fixtures.

218

4. user (2026-06-10T16:13:34.949Z)

1 # Roadmap

2

3 Next steps for the modernised project, grouped by effort and grounded in the

4 current Apps Script / Gmail platform constraints.

5

6 ## Platform constraints that shape the design

7

8 For consumer (gmail.com) accounts:

9

10 - ****6 minutes**** max per execution; ****90 minutes/day**** total trigger runtime;

11 ****20 triggers**** per script per user; **~**20,000**** Gmail read calls/day.

12 ([Apps Script quotas](https://developers.google.com/apps-script/guides/services/quotas))

13 - There is ****no clearly documented daily Drive file-create quota****, so

14 `MAX\_FILES\_ADD\_PER\_DAY` (240) is a self-imposed heuristic, not an API limit.

15 - Gmail's

16 [`users.history.list`](https://developers.google.com/workspace/gmail/api/guides/sync)

17 with a stored `startHistoryId` is the purpose-built incremental sync (~2 quota

18 units vs 5 for a list call) and returns only changes. A `historyId` can expire

19 (HTTP 404 ? fall back to full sync), so it complements the date-window

20 backfill rather than replacing it.

21 - clasp can deploy from CI using a `CLASPRC\_JSON` secret (contents of

22 `~/.clasprc.json`); service accounts do ****not**** work with the Apps Script API,

23 so use a dedicated owner account's token.

24 ([clasp #225](https://github.com/google/clasp/issues/225))

25

26 ## Quick wins

27

28 - [x] ****Cover the orchestration shells with tests.**** `sync.ts` / `ui.ts` had no

29 coverage ? the trickiest logic (trigger fan-out, quota reset, catch-up loop,

30 `processInput` state reset). *(done in this branch; see `tests/sync.test.ts`,

31 `tests/ui.test.ts`)*

32

33 - [x] ****Raise the per-run wall-clock budget.**** Was capped at 1 minute against a

```

32     6-minute ceiling, making backfill ~6× slower than necessary. Now
33     `MAX_RUNTIME_MS` (5 min) in `src/config.ts`, threaded through `hasTime`.
34 - [x] **Guard against trigger leakage.** A `LockService` script lock prevents
35     concurrent runs from stacking triggers, deletion is scoped to the sync
36     handler, and a cap prunes any excess. *(done in this branch; `src/sync.ts`)*
37 - [x] **Add a Jest `coverageThreshold`** so CI fails if coverage regresses.
38     *(done; `jest.config.js`, gated at 85% stmts/lines/funcs, 75% branches,
39     `entry.ts` bundler glue excluded)*
40
41     ## Medium
42
43 - [x] **CI/CD auto-deploy on merge to `master`.** A gated `deploy` job writes
44     `CLASPRC_JSON` ? `~/clasprc.json` and runs `clasp push` / `clasp deploy`
45     against the committed `.clasp.json`. *(done; `.github/workflows/ci.yml`.
46     Requires only the `CLASPRC_JSON` repo secret; skips cleanly if unset.)*
47 - [x] **Cross-run dedup by message/attachment ID.** A bounded FIFO of processed
48     Gmail message ids (`src/processed.ts`, capped at `MAX_PROCESSED_IDS`) is
49     persisted in user properties; `updateDrive` skips messages already in the set
50     and reports newly-completed ids back to `sync.ts`. A message is only marked
51     done when all its attachments were copied cleanly (not truncated by the daily
52     cap or a transient error), so re-scanned windows skip the Drive
53     `getFilesByName` probe. *(done; `src/processed.ts`, `src/drive.ts`,
54     `src/sync.ts`, `src/state.ts`)*
55 - [x] **Observability.** Level-prefixed structured logging (`src/logger.ts`),
56     per-day run stats accumulator (`src/stats.ts` ? runs/files/errors/last error,
57     persisted via `PROP_RUN_STATS`), and an opt-in daily summary
58     (`src/notify.ts`) logged on each new-day rollover. The summary email is OFF by
59     default; enabling it needs `ENABLE_SUMMARY_EMAIL` + the `script.send_mail`
60     scope. *(done)*
61
62     ## Larger / architectural
63
64 - [x] **Hybrid incremental sync.** Keep the date-window scan for the initial
65     backfill; once caught up (`isToday`), switch the steady state to the Gmail
66     advanced service + `history.list`/`startHistoryId`, storing `historyId` in
67     properties and falling back to full sync on 404. *(done; pure logic in
68     `src/historySync.ts`, advanced-service shell in `src/gmailHistory.ts`, wired
69     into `src/sync.ts`. Gated behind `ENABLE_INCREMENTAL_SYNC` (OFF by default)
70     for gradual rollout over the proven backfill. The Gmail advanced service is
71     declared in `appsscript.json`; no new OAuth scope ? `gmail.readonly` covers
72     history reads. Backfill captures the mailbox `historyId` before processing so
73     mail arriving mid-run is still caught; expiry (HTTP 404) clears the cursor and
74     rewinds to a bounded date-window re-scan.)*
75 - [ ] **More structure options & Shared Drive support** ? by-sender, nested
76     `Year/Month`, label-driven folders; currently My Drive only.
77
78     ## Open-source & auditability readiness
79
80     Concrete prep so the [auditability invariant](docs/ARCHITECTURE.md#auditability-
81     invariant-binding)
82     is real, not aspirational, before/when the repo is opened up.
83
84 - [x] Root `LICENSE` file (MIT, matching the source headers).
85 - [x] Egress guard ? a test that fails CI if `src/` makes any network call
86     (`tests/auditability.test.ts`). Update it deliberately when the user-endpoint
87     egress lands (assert the allowlisted destination, don't remove the guard).
88 - [ ] `SECURITY.md` ? how to report issues + the current data-flow guarantees
89     (today: reads Gmail, writes the user's own Drive, **no egress**).
90 - [ ] **"Deploy your own from source" guide ? self-host the GAS project end to
91     end, so the trust claim is "run the code you read."
92 - [ ] **Reproducible / verifiable build** ? pin the toolchain and document how to
93     diff a deployed `dist/Code.js` against a build of a given commit.
94 - [ ] `CONTRIBUTING.md` carrying the auditability PR checklist.

```

5. user (2026-06-10T16:13:35.128Z)

```
1      # Gmail Attachments ? Google Drive
2
3      A Google Apps Script that copies your Gmail attachments into a `GmailAttachments`
4      folder in Google Drive and keeps it updated as new mail arrives. You choose how
5      the attachments are organised (by content type, by month, or by year) and how
6      far back the first sync should reach.
7
8      Originally a 2013 Chrome Web Store app; this is a modernised rebuild on the
9      **V8 runtime** with a TypeScript source, an esbuild build, and a local Jest test
10     suite. The web app runs **as each accessing user** (`executeAs: USER_ACCESSING`),
11     so it can serve just you, a small group, or a Google Workspace org ? each
12     person's attachments are copied within their own account. See
13     [Multi-user deployment / publishing](#multi-user-deployment--publishing).
14
15     > **Where this is headed:** attachment archiving is the hardened core of a larger
16     > goal ? turning email into a structured, queryable record of one's life
17     > (receipts, statements, travel, ?) via an ingest ? classify ? extract ? handoff
18     > pipeline, with domain processing isolated in downstream services. See
19     > [`docs/VISION.md`](./docs/VISION.md) and
20     [`docs/ARCHITECTURE.md`](./docs/ARCHITECTURE.md).
21
22     ## How it works
23
24     - `doGet` serves a setup form (`html/index.html`).
25     - `processInput` stores your choices and schedules the first background run.
26     - `syncUserDrives` is a self-rescheduling time-based trigger that walks Gmail in
27       30-day windows, copying allowed attachments into Drive while respecting a
28       daily file cap (`MAX_FILES_ADD_PER_DAY`) and a per-run wall-clock budget.
29
30     State lives in `PropertiesService` (user properties). Existing Drive files are
31     never modified ? re-running setup only resets the script's own bookkeeping.
32
33     ## Project layout
34
35     | Path | Purpose |
36     | --- | --- |
37     | `src/` | TypeScript source. Pure logic (`query`, `naming`, `quota`, `contentTypes`) is
38     separated from the Google-service shells (`drive`, `sync`, `ui`, `state`). |
39     | `src/entry.ts` | Bundle entry; esbuild exposes `doGet` / `processInput` /
40     `syncUserDrives` as globals. |
41     | `html/index.html` | The setup form (pushed to Apps Script as `index`). |
42     | `appsscript.json` | Manifest: V8 runtime, OAuth scopes, web-app settings. |
43     | `tests/` | Jest unit tests. |
44     | `dist/` | Build output that `clasp` pushes (git-ignored). |
45
46     ## Develop
47
48     ```bash
49     npm install
50     npm run typecheck    # tsc --noEmit
51     npm run lint         # eslint
52     npm test             # jest
53     npm run build        # bundle src -> dist/Code.js (+ manifest + html)
54     ```
55
56     ## Deploy (requires your Google account)
57
58     1. `npm install -g @google/clasp && clasp login`
59     2. The project is already linked via the committed `.clasp.json` (`scriptId` +
60        `rootDir: dist`). To target a different project, edit `scriptId`.
61     3. `npm run push` ? builds and `clasp push`es `dist/`.
62     4. In the Apps Script editor, deploy as a **Web app**. The manifest already sets
63        *Execute as: User accessing* and *Who has access: Anyone*, so each visitor
```

```

61     runs against their own account. Open the URL and submit the form.
62 5. Set `timeZone` in `appsscript.json` to your own zone ? the Gmail
63    `before:`/`after:` day boundaries follow it.
64
65 > Using it only for yourself? It still works as-is ? you're simply the only
66 > user. To let others in, follow the runbook below.
67
68 ### Auto-deploy from CI
69
70 Pushing to `master` runs the `deploy` job in `.github/workflows/ci.yml`, which
71 `clasp push`es and `clasp deploy`s the build. It needs a single repository
72 secret:
73
74 - `CLASPRC_JSON` ? the full contents of `~/.clasprc.json` after `clasp login`
75   (generate it from a dedicated owner account; the token refreshes over time).
76
77 The `scriptId` lives in the committed `.clasp.json` (it is not sensitive ? it
78 appears in the web-app URL and grants nothing without the OAuth token). If the
79 secret is unset the job logs a warning and skips, so forks/CI without
80 credentials stay green.
81
82 ### OAuth scopes
83
84 `gmail.readonly` (we only read mail), `drive` (create/enumerate folders and
85 files), and `script.scriptapp` (manage the background triggers).
86
87 ## Observability
88
89 Each run logs structured, level-prefixed lines (`[INFO]` / `[ERROR]` / `[DEBUG]`)
90 to the Apps Script execution log / Stackdriver, and accumulates per-day stats
91 (runs, files added, errors, last error). On the first run of a new calendar day
92 the previous day's summary is logged.
93
94 ### Optional daily summary email
95
96 Off by default. To receive the daily summary by email:
97
98 1. Set `ENABLE_SUMMARY_EMAIL = true` (and optionally `SUMMARY_EMAIL_RECIPIENT`)
99   in `src/config.ts`.
100 2. Add the mail-send scope to `appsscript.json` `oauthScopes`:
101   `https://www.googleapis.com/auth/script.send_mail`
102 3. Rebuild and redeploy; you'll be prompted to re-consent to the new
103   "send email as you" permission.
104
105 ## Multi-user deployment / publishing
106
107 The app is already wired for multiple users (runs as the accessing user, with
108 all state in *their* `getUserProperties`, a per-user lock, and per-user
109 triggers). What it takes to actually let other people in depends on the audience
110 ? and this project deliberately targets the **no-verification** zone: a small
111 named group and/or a single Workspace org. That avoids Google's brand /
112 restricted-scope verification **and** the annual paid third-party CASA security
113 assessment that a fully public launch with `gmail.readonly` + `drive` would
114 require.
115
116 ### One-time Google Cloud setup
117
118 1. **Use a standard Cloud project** (the default Apps Script one can't host a
119   managed consent screen). Create a project in the
120   [Cloud Console](https://console.cloud.google.com/), then in the Apps Script
121   editor: **Project Settings ? Google Cloud Platform (GCP) Project ? Change
122   project** and paste the project number. *This switch is permanent.*
123 2. **Enable APIs** in that project: **Gmail API** (required by the advanced Gmail
124   service this app uses) and **Google Drive API**.
125 3. **Configure the OAuth consent screen / Branding**: app name

```

```

126     (`Gmail Attachments to Drive`), a user-support email, a developer-contact
127     email, and ? recommended ? a homepage and a privacy policy URL. A ready
128     policy lives at [PRIVACY.md](./PRIVACY.md); host it (e.g. GitHub Pages) and
129     add that domain under Authorized domains.
130
131     ### Pick an audience (same build, different consent config)
132
133     | | Small group | Workspace org |
134     | --- | --- | --- |
135     | User type | External, status Testing | Internal |
136     | Who can use it | Up to 100 Google accounts you add as Test users | Everyone
in your Workspace org |
137     | Consent screen | Shows a "Google hasn't verified this app" warning (expected) | No
warning |
138     | Caveat | Test users re-consent ~weekly (7-day refresh-token expiry) | Org-only;
not usable by external gmail.com accounts |
139     | Verification / CASA | None | None |
140
141     Then deploy (the CI clasp push`/`deploy` flow is unchanged) and share the
142     /exec` web-app URL. Note: switching the GCP project / executeAs` forces
143     everyone ? including you ? to re-authorize once.
144
145     ### Going fully public (out of scope here)
146
147     Letting anyone use it (External + Production) requires brand verification,
148     restricted-scope verification, domain verification, and an annual CASA
149     security assessment(~$1?5k+, 6?16 weeks). If you ever go there, first narrow
150     drive` ? drive.file` (the app only touches files it creates) to shrink the
151     review surface ? though gmail.readonly` remains restricted regardless.
152
153     ## License
154
155     MIT ? see headers in source files.
156

```

6. user (2026-06-10T16:13:41.318Z)

```

1     # Privacy Policy ? Gmail Attachments to Drive
2
3     _Last updated: 2026-05-31_
4
5     Gmail Attachments to Drive ("the app") is a Google Apps Script web app that
6     copies attachments from your Gmail into a GmailAttachments` folder in your own
7     Google Drive.
8
9     ## What the app accesses
10
11     When you authorize the app, it requests permission to:
12
13     - Read your Gmail (gmail.readonly`) ? to find messages with attachments and
14       read those attachments so they can be copied.
15     - Manage your Google Drive (drive`) ? to create the GmailAttachments`
16       folder and its sub-folders and to write the copied attachment files.
17     - Run in the background (script.scriptapp`) ? to schedule the periodic sync
18       on your behalf.
19
20     ## How your data is used
21
22     - The app runs entirely within your own Google account, as you
23       (the web app is deployed with executeAs: USER_ACCESSING`).
24     - Attachments are copied from your Gmail to your Drive only. Your email
25       content and files are never sent to, stored on, or transmitted through any
26       third-party server. The author of the app has no access to your data.
27     - The only data the app stores is small bookkeeping state (your folder-structure
28       choice, a sync cursor, counters, and recently-processed message IDs) in

```

```

29     **Google's per-user `PropertiesService`, inside your account.
30
31     ## Data sharing and retention
32
33     - **No data is shared** with the developer or any third party.
34     - Bookkeeping state lives in your account for as long as the app is installed.
35       Re-running setup clears it. Files already copied to your Drive are yours and
36       are never modified or deleted by the app.
37
38     ## Revoking access
39
40     You can revoke the app's access at any time at
41     [myaccount.google.com/permissions](https://myaccount.google.com/permissions).
42     This stops future syncs; attachments already copied to your Drive remain.
43
44     ## Contact
45
46     Questions or issues:
47     [github.com/avidullu/gmail-attachments-to-gdrive](https://github.com/avidullu/gmail-
48     attachments-to-gdrive)
49     (open an issue).

```

7. user (2026-06-10T16:13:42.264Z)

```

1      {
2        "timeZone": "America/Los_Angeles",
3        "runtimeVersion": "V8",
4        "exceptionLogging": "STACKDRIVER",
5        "dependencies": {
6          "enabledAdvancedServices": [
7            {
8              "userSymbol": "Gmail",
9              "serviceId": "gmail",
10             "version": "v1"
11           }
12         ]
13       },
14       "oauthScopes": [
15         "https://www.googleapis.com/auth/gmail.readonly",
16         "https://www.googleapis.com/auth/drive",
17         "https://www.googleapis.com/auth/script.scriptapp"
18       ],
19       "webapp": {
20         "access": "ANYONE",
21         "executeAs": "USER_ACCESSING"
22       }
23     }
24

```

8. user (2026-06-10T16:13:42.731Z)

```

1      {
2        "name": "gmail-attachments-to-gdrive",
3        "version": "2.0.0",
4        "description": "Google Apps Script that syncs Gmail attachments into Google Drive.",
5        "license": "MIT",
6        "private": true,
7        "scripts": {
8          "build": "node esbuild.config.mjs",
9          "typecheck": "tsc --noEmit",
10         "lint": "eslint src tests",
11         "test": "jest",
12         "test:coverage": "jest --coverage",
13         "push": "npm run build && clasp push",

```

```

14     "deploy": "npm run build && clasp deploy"
15 },
16 "devDependencies": {
17     "@google/clasp": "^2.4.2",
18     "@types/google-apps-script": "^1.0.83",
19     "@types/jest": "^29.5.12",
20     "@types/node": "^20.12.7",
21     "esbuild": "^0.21.5",
22     "eslint": "^9.5.0",
23     "jest": "^29.7.0",
24     "ts-jest": "^29.1.4",
25     "typescript": "^5.4.5",
26     "typescript-eslint": "^8.0.0"
27 }
28 }
29

```

9. user (2026-06-10T16:13:43.041Z)

```

1  MIT License
2
3  Copyright (c) 2013 Avi Dullu
4
5  Permission is hereby granted, free of charge, to any person obtaining a copy
6  of this software and associated documentation files (the "Software"), to deal
7  in the Software without restriction, including without limitation the rights
8  to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
9  copies of the Software, and to permit persons to whom the Software is
10 furnished to do so, subject to the following conditions:
11
12 The above copyright notice and this permission notice shall be included in
13 all copies or substantial portions of the Software.
14
15 THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
16 IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
17 FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
18 AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
19 LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
20 OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN
21 THE SOFTWARE.
22

```

10. user (2026-06-10T16:13:49.804Z)

No files found

11. user (2026-06-10T16:13:50.097Z)

No files found

12. user (2026-06-10T16:13:50.456Z)

```

1  import { readFileSync, readdirSync } from "fs";
2  import { join } from "path";
3
4  /*
5   * Enforces the auditability invariant (docs/ARCHITECTURE.md): this repo is the
6   * only code that touches a user's Gmail, and it makes NO network egress today.
7   *
8   * The ONLY sanctioned way data may ever leave a user's account is a deliberate,
9   * reviewed REDACT/MINIMIZE -> user's-own-endpoint path. When that lands, update
10  * this guard to assert the single allowlisted destination ? do not delete it.
11  */
12

```

```

13 // Apps Script's only outbound-network primitive.
14 const FORBIDDEN = /UrlFetchApp/;
15
16 function tsFiles(dir: string): string[] {
17   return readdirSync(dir, { withFileTypes: true }).flatMap((entry) => {
18     const full = join(dir, entry.name);
19     if (entry.isDirectory()) return tsFiles(full);
20     return entry.name.endsWith(".ts") ? [full] : [];
21   });
22 }
23
24 describe("auditability invariant", () => {
25   it("src/ makes no outbound network calls (no UrlFetchApp)", () => {
26     const offenders = tsFiles("src").filter((file) =>
27       FORBIDDEN.test(readFileSync(file, "utf8")),
28     );
29     expect(offenders).toEqual([]);
30   });
31 });
32

```

13. user (2026-06-10T16:13:58.197Z)

```

types
config.ts
contentTypes.ts
drive.ts
entry.ts
gmailHistory.ts
historySync.ts
logger.ts
naming.ts
notify.ts
processed.ts
query.ts
quota.ts
state.ts
stats.ts
sync.ts
ui.ts
types\gmailAdvanced.d.ts
auditability.test.ts
contentTypes.test.ts
drive.test.ts
gmailHistory.test.ts
historySync.test.ts
naming.test.ts
notify.test.ts
processed.test.ts
query.test.ts
quota.test.ts
setup.ts
stats.test.ts
sync.test.ts
syncIncremental.test.ts
ui.test.ts
ARCHITECTURE.md
VISION.md

```

14. user (2026-06-10T16:14:07.085Z)

```

C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\notify.ts:37:
MailApp.sendEmail(to, "Gmail?Drive sync summary " + stats.day, summary);

```

15. user (2026-06-10T16:14:08.874Z)

No matches found

16. user (2026-06-10T16:14:09.033Z)

```
1      import {
2          ENABLE_SUMMARY_EMAIL,
3          SUMMARY_EMAIL_RECIPIENT,
4      } from "./config";
5      import { logError, logInfo } from "./logger";
6      import { dayKey, formatSummary } from "./stats";
7      import * as state from "./state";
8
9      /**
10       * Summarises the previous day's activity the first time a sync runs on a new
11       * calendar day. Always logs the summary; emailing is opt-in
12       * (ENABLE_SUMMARY_EMAIL, which requires the script.send_mail scope).
13       *
14       * `now` is injected for testability.
15       */
16      export function maybeSendDailySummary(now: Date): void {
17          const today = dayKey(now);
18          if (state.getSummaryLastSent() === today) {
19              return; // already summarised since the last day rolled over
20          }
21
22          // Summarise whatever the persisted accumulator holds (the prior day's run).
23          const stats = state.getRunStatsRaw();
24          if (!stats || stats.day !== today) {
25              // Nothing from a prior day to report yet; arm for the next rollover.
26              state.setSummaryLastSent(today);
27              return;
28          }
29
30          const summary = formatSummary(stats);
31          logInfo(summary);
32
33          if (ENABLE_SUMMARY_EMAIL) {
34              try {
35                  const to = SUMMARY_EMAIL_RECIPIENT || Session.getEffectiveUser().getEmail();
36                  if (to) {
37                      MailApp.sendEmail(to, "Gmail?Drive sync summary " + stats.day, summary);
38                  }
39              } catch (e) {
40                  logError("Failed to send summary email: " + e);
41              }
42          }
43
44          state.setSummaryLastSent(today);
45      }
46
```

17. user (2026-06-10T16:14:17.491Z)

48: **Goal:** pull messages and attachments from Gmail and Outlook into a local staging area.

50: **Gmail:** [Gmail API](<https://developers.google.com/gmail/api>) over OAuth2. Read-only scope (`gmail.readonly`). Supports server-side search queries (`has:attachment filename:pdf from:hdfc newer\_than:90d`) which is far cheaper than pulling everything and filtering locally. Has good rate limits for personal use.

52: **Why not IMAP:** worse search, no labels, harder OAuth, and we lose Gmail's powerful query syntax. Use the native APIs.

54: Mechanically straightforward. The main design questions are: **how is "financial document" detected** (sender allowlist? subject patterns? attachment heuristics? hybrid?), and **how is incremental sync state tracked** (Gmail `historyId` / Graph `deltaLink`).

```
148:> **Gmail ? one specific bank's monthly statements ? unlocked ? parsed into `transactions` ?
stored in SQLite ? a CLI command that lists last month's transactions.**
184:$ 7 proposes "Gmail + one bank ? CLI". Confirm with user before building.
186:> **Resolved (2026-05-30): Build both tracks in parallel** ? the promo-corpus
parse?store?query spine *and* the Gmail ingestion plumbing, as two decoupled tracks that
converge at the first bank slice; first bank = **HDFC**. This refines $ 7 (which proposed a
single end-to-end bank slice first). See [ `docs/ROADMAP.md` ](./docs/ROADMAP.md) §§ 1 and 4.
234:[Omitted long matching line]
```

18. user (2026-06-10T16:14:21.640Z)

No matches found

19. user (2026-06-10T16:14:29.981Z)

```
15:> **Track B:** Gmail OAuth2 + search + attachment download ?; the `muneem fetch` CLI (B.4) is
not
115:  parse ? store ? query                fetch ? stage ? catalog                fully decoupled)
131:Tracks A and B are genuinely independent: A needs no OAuth and no secrets; B needs no
parsing. As a single developer you can interleave them in any order ? neither blocks the other.
**Both must land before Milestone C**, where a fetched-and-unlocked HDFC PDF flows through the
parser for the first time.
157:### Track B ? Gmail ingestion · PARTIAL (OAuth + search + download ?; `muneem fetch` B.4
pending)
163:- [ ] **B.3 (S)** Incremental sync: persist Gmail `historyId`; on re-run, fetch only what's
new. Idempotent (re-running never duplicates `documents`, keyed by message id + attachment
hash).
164:- [ ] **B.4 (S)** Catalog fetched attachments as `documents` rows (status
`locked`/`unparsed`), with sha256 + staging path. `muneem fetch --since 90d` is the entry point.
165:- **Exit:** `muneem fetch` populates the staging dir + `documents` rows from a narrow real-
Gmail query. (Manual/local verification ? not CI; it touches a real account.)
169:> A fetched HDFC statement gets unlocked and parsed into real transactions. This is the
moment every stage runs once on real data the user cares about.
306:The DESIGN.md § 7 slice, hardened: **`muneem fetch` pulls HDFC statements from Gmail ?
unlocks them with the hybrid password strategy ? the HDFC parser writes normalized
`transactions` to SQLite ? `muneem txns list` shows last month's activity ? all locally, with
redacted logs, idempotent re-runs, green CI on synthetic fixtures, and a written evaluation of
where to go next.** The promo "forgotten deals" feature ships alongside as the Track A
deliverable.
```

20. user (2026-06-10T16:14:53.923Z)

```
4:import { logDebug, logError } from "./logger";
38:      logDebug("Found existing folder " + name);
42:  logDebug("Creating new folder " + name);
96:  logDebug("Already present folders: " + existingFolders.size);
112:      logDebug("Skipping already-processed message " + messageId);
129:      logDebug("Rejected file of type: " + contentType);
142:      logError("Could not resolve directory; using top-level folder.");
166:      logError("Attachment copy failed (message " + messageId + "): " + e);
```

21. user (2026-06-10T16:14:54.166Z)

```
1      import { ENABLE_DEBUGGING } from "./config";
2
3      /**
4       * Minimal structured logger. Each line is prefixed with a level so the Apps
5       * Script execution log / Stackdriver can be filtered (e.g. `severity:ERROR`).
6       * Kept dependency-free and side-effect-only so callers stay simple.
7       */
8      function emit(level: "DEBUG" | "INFO" | "ERROR", data: unknown): void {
9          Logger.log([" " + level + " ] " + String(data));
10     }
11
```

```
12     /** Verbose log, only emitted when ENABLE_DEBUGGING is on. */
13     export function logDebug(data: unknown): void {
14         if (ENABLE_DEBUGGING) {
15             emit("DEBUG", data);
16         }
17     }
18
19     /** Informational log; always emitted. */
20     export function logInfo(data: unknown): void {
21         emit("INFO", data);
22     }
23
24     /** Error log; always emitted and surfaces in Stackdriver as an error. */
25     export function logError(data: unknown): void {
26         emit("ERROR", data);
27     }
28
```

22. user (2026-06-10T16:18:27.117Z)

Structured output provided successfully

23. assistant (2026-06-10T16:18:33.098Z)

Audit complete. The structured report has been returned via StructuredOutput.